

Day 4 - STL Containers

You almost never write your own dynamic array or hash map in a game. You reach for the STL. Knowing which container fits which job (and the cost of each) is the difference between code that scales to thousands of entities and code that stutters. Today is about choosing well and iterating cleanly.

Learning objectives

By the end of today you can:

- ☐ Use `std::vector` and explain its contiguous-storage / growth model.
- ☐ Use `std::map` for an ordered key→value table and know its O(log n) cost.
- ☐ Use `std::set` for unique, sorted membership and deduplication.
- ☐ Traverse containers with iterators and range-based for.
- ☐ Justify a container choice from access pattern and complexity, not habit.

Concepts

std::vector

A growable, contiguous array, and the default container. `push_back` is amortised O(1); indexing is O(1); the contiguous layout is cache-friendly. If you know the final size, `reserve` up front to avoid reallocations.

```
std::vector<Entity> entities;
entities.reserve(1000);            // one allocation instead of many
entities.push_back(Entity{"orc"});
Entity& first = entities[0];       // O(1) random access
```

❌ **Common Mistake:** holding a pointer/reference/iterator into a vector across a `push_back`. A reallocation moves the elements and leaves it dangling (see **Iterators**).

✅ **Good Practice:** store an index instead, or re-take the reference after the insert; `reserve()` up front when you know the final size.

std::map

A balanced binary tree mapping keys to values: lookup, insert, and erase are all O(log n), and iteration walks the keys in *sorted* order for free. Reach for it when you need a key→value table *and* ordered traversal or range queries. (When you don't care about order, `std::unordered_map` is the hashed sibling: average O(1) lookup, but no ordering. That's the trade-off.)

```
std::map<int, Entity> byId;          // ordered id -> entity table
byId[42] = e;                         // operator[] inserts or overwrites
auto it = byId.find(42);              // find() looks up without inserting
if (it != byId.end()) use(it->second);

for (const auto& [id, e] : byId)      // walks ids in ascending order
    std::cout << id << ' ' << e.name << '\n';
```

❌ **Common Mistake:** `map[key]` on a missing key quietly *inserts* a default entry. A read grows the map, and later iterations see a key nobody added.

✅ **Good Practice:** use `find` / `count` when you only mean to look up, and keep `operator[]` for when you really do want insert-or-overwrite.

std::set

A sorted collection of *unique* keys, a `std::map` with keys but no values. Inserting a duplicate is a no-op, iteration is sorted, and membership tests (`count` / `find`) are O(log n). Reach for it to deduplicate, to keep a sorted set of ids, or to answer "have I already seen this?".

```
std::set<int> ids;
ids.insert(7);
ids.insert(7);                        // duplicate: no-op, still one 7
ids.insert(2);
bool seen = ids.count(7);             // 1 if present, 0 if not
for (int id : ids) std::cout << id << '\n';   // prints 2 then 7, sorted
```

❌ **Common Mistake:** using a `std::vector` plus a manual sort/dedup pass when uniqueness and order are the whole point. That's hand-written code, and a bug surface, to re-earn what a container already guarantees.

✅ **Good Practice:** reach for `std::set`, or `std::unordered_set` when order doesn't matter.

Iterators

An iterator is a generalised pointer into a container; `begin()` points at the first element, `end()` one past the last. Prefer range-based for, but reach for explicit iterators when you need to erase while traversing. **Invalidation** is the classic bug: mutating a container can invalidate iterators/pointers into it.

```
// Erase-while-iterating: erase() returns the next valid iterator
for (auto it = v.begin(); it != v.end(); ) {
    if (shouldRemove(*it)) it = v.erase(it);   // don't ++ here
    else                   ++it;
}
```

❌ **Common Mistake:** `++it` after `erase(it)`. The erased iterator is already invalid, so incrementing it is undefined behaviour.

✅ **Good Practice:** advance with the iterator `erase` returns, and only `++it` on the branch that didn't erase.

Range-based for

`for (const auto& x : container)` is the clean default: `auto` deduces the element type, `&` avoids copying each element, `const` signals you won't modify it.
Drop `const` (keep `&`) when you *do* intend to modify elements in place.

```
for (const auto& e : entities) render(e);   // read-only, no copies
for (auto& e : entities)       e.tick(16);  // modify in place
```

❌ **Common Mistake:** `for (auto x : container)`. Omitting `&` copies every element on every iteration, and writes to `x` are lost.

✅ **Good Practice:** default to `for (const auto& x : container)`; drop the `const` (keep the `&`) only when you modify in place.

Choosing a container

Need	Reach for	Why
Default sequence, index access, iterate a lot	`std::vector`	Contiguous, cache-friendly, O(1) index
FIFO processing (BFS, events, turns)	`std::queue`	Purpose-built push-back / pop-front
LIFO / undo stack	`std::stack`	Purpose-built push/pop one end
Fast key → value lookup, order irrelevant	`std::unordered_map`	Average O(1)
Key → value with sorted iteration / ranges	`std::map`	Ordered, O(log n)
Membership test, unique elements	`std::unordered_set` / `std::set`	Same trade-off as the maps
Frequent insert/erase in the *middle*	`std::list`	O(1) splice, but no random access

Default to `std::vector`; justify anything else from the access pattern, not habit. The three you'll practise today, `std::vector`, `std::map`, and `std::set`, are the ones you'll reach for most. The rest of the table is the wider menu to recognise.

Exercises

Each exercise gives you the exact data and the exact output a correct program prints, but **not** which container to use. Read what the data has to *do* (Keep insertion order? Look up by key? Stay unique and sorted?), then choose the fitting container from **Concepts**, using the *Choosing a container* table as your decision guide, and be ready to justify it. That choice is the skill this day grades. The same four entities (`orc`, `bat`, `boss`, `rat`) recur all day, so you'll recognise them from one exercise to the next.

ex00 - Build and prune the roster

.	.	.
💢	**Difficulty**	⭐⭐⭐
🧠	**Objectives**	*Store the four entities in the order you add them, print the roster, then remove every entity with `hp <= 5` while traversing, without invalidating the walk.*
📁	**Turn in directory**	`ex00/`
❗	**Turn in files**	**Entity.hpp, main.cpp, Makefile**
🛠️	**Skills required**	*Matching a container to the access pattern, safely erasing while iterating (use the iterator `erase` returns), range-based traversal.*
🔍	**Search keywords**	*C++ STL container choice, iterator invalidation, erase while iterating, erase returns next iterator, range-based for*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<string>`, `<vector>`, `<map>`, `<set>`

- **Goal:** Store the four entities below in a container that keeps them in the order you add them and lets you scan front-to-back. Print the roster in that order, then remove every entity with `hp <= 5` **while you traverse**, without invalidating the traversal (see the *Iterators* concept). Binary name: `roster`.
- **Done when:** The output matches below, only `rat` (hp 4) is gone with the rest in their original order, and no iterator or reference is used after a removal invalidates it.

Given:

```
struct Entity { int id; std::string name; int hp; };

// seed, in this order:
{7, "orc", 30}, {2, "bat", 8}, {42, "boss", 300}, {13, "rat", 4}
```

Output example:

```
$ ./roster
-- roster --
7 orc (hp 30)
2 bat (hp 8)
42 boss (hp 300)
13 rat (hp 4)
-- survivors --
7 orc (hp 30)
2 bat (hp 8)
42 boss (hp 300)
```

You needed insertion order plus in-place removal that doesn't break the walk, which is the job of the default sequence container.

ex01 - Look an entity up by id

.	.	.
💢	**Difficulty**	⭐⭐⭐
🧠	**Objectives**	*Store the four entities so you can look one up directly by `id` without inserting on a miss, iterate them in ascending id order using the container's own ordering, and justify the choice in `NOTES.md`.*
📁	**Turn in directory**	`ex01/`
❗	**Turn in files**	**Entity.hpp, main.cpp, NOTES.md, Makefile**
🛠️	**Skills required**	*Matching a container to a key-lookup pattern, looking up without inserting on a miss, iterating in the container's native order, justifying the choice in writing.*
🔍	**Search keywords**	*C++ key lookup container, find vs operator[] insert, ordered iteration, associative container, container choice justification*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<string>`, `<vector>`, `<map>`, `<set>`

- **Goal:** Store the four entities so you can look one up **directly by its `id`**. Look up a present id (`42`) and a missing id (`99`) with a lookup that does **not** create an entry when the id is absent (the indexing `operator[]` would, so avoid it). Then walk the whole container and print every id in **ascending order, using the container's own ordering, with no manual sort**. In `NOTES.md`, name the container you chose and why. Binary name: `roster`.
- **Done when:** The two lookups match below, iterating prints the ids ascending as shown without a sort of your own, and `NOTES.md` justifies the choice.

Given:

```
struct Entity { int id; std::string name; int hp; };

// seed:
{7, "orc", 30}, {2, "bat", 8}, {42, "boss", 300}, {13, "rat", 4}
```

Output example:

```
$ ./roster
id 42: boss
id 99: (not found)
```

```
-- entities by id --
2 bat
7 orc
13 rat
42 boss
```

ex02 - Unique spawns

.	.	.
💢	**Difficulty**	⭐⭐
🧠	**Objectives**	*Collect the spawn stream into a container that keeps each id at most once and sorted, print the unique ids and their count, then answer two membership questions with the container's own test.*
📁	**Turn in directory**	`ex02/`
❗	**Turn in files**	**main.cpp, Makefile**
🛠️	**Skills required**	*Matching a container to a uniqueness-plus-ordering requirement, using a built-in membership test, avoiding a manual dedup or sort pass.*
🔍	**Search keywords**	*C++ unique sorted collection, membership test count find, deduplicate without sorting, associative container*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<string>`, `<vector>`, `<map>`, `<set>`

- **Goal:** The spawn stream below fires ids, some more than once. Collect them into a container that stores each id **at most once** and keeps them **sorted**, then print the unique ids and how many there are. Finally answer two membership questions, is `42` among them and is `99`, with the container's **own** membership test rather than a manual scan. Binary name: `spawns`.
- **Done when:** Duplicates collapse to one each, the ids print sorted with count `4`, and the two membership checks match below.

Given:

```
// spawn stream (note the repeats):
7, 2, 7, 42, 13, 2, 42
```

Output example:

```
$ ./spawns
-- spawned (unique, sorted) --
2
7
13
42
count: 4
is 42 spawned? yes
is 99 spawned? no
```

Seven ids go in and four come out: unique keys, kept in sorted order, with a built-in membership test and no dedup-or-sort pass of your own. That exact trio of properties is one container's whole reason to exist.

Constraints (every exercise)

- Compiles with `c++ -Wall -Wextra -Werror -std=c++17`.
- Prefer range-based for; use explicit iterators only where needed.
- No raw owning `new` / `delete`; let containers own.
- Follows Day 1 naming and header conventions.

How you're evaluated

An evaluation will be automatically scheduled with a peer and start at 6pm. Submit all the work you have done so far even if its not 100% complete.

Turning in your work

1. Create a new repository on GitHub named `gauntlet-day-04`, and keep it **public**. A private repo is one your evaluator cannot clone, and the evaluation cannot start without the code.
2. Commit today's work there, one directory per exercise (`ex00/`, `ex01/`, ...) exactly as each turn-in table says, `Makefile` included.
3. Push before the evaluation. Whatever sits on the default branch when the evaluation starts is what gets read.
4. When the evaluation starts, give your evaluator the link to the repository.

AI-written code is an immediate disqualification. Look things up and ask an AI what something does, that is a tool. Pasting in code you did not write is not.